

# k-Subgraph Isomorphism Problem

Mateusz Osik, Mateusz Polis, Mateusz Nedzi

December 2025

## Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Problem Statement</b>                                               | <b>1</b> |
| <b>2</b> | <b>Definition of Graph Size</b>                                        | <b>2</b> |
| <b>3</b> | <b>Metric in the Space of Graphs</b>                                   | <b>2</b> |
| <b>4</b> | <b>Exact Minimal Extension for <math>k</math>-Subgraph Isomorphism</b> | <b>3</b> |
| <b>5</b> | <b>Polynomial Approximation</b>                                        | <b>6</b> |
| <b>6</b> | <b>Experiments</b>                                                     | <b>9</b> |
| 6.1      | Experimental Setup . . . . .                                           | 9        |
| 6.2      | Time Complexity Analysis . . . . .                                     | 9        |
| 6.3      | Solution Quality Analysis . . . . .                                    | 10       |
| 6.4      | Trade-off Summary . . . . .                                            | 11       |

## 1 Problem Statement

The task focuses on the *k-Subgraph Isomorphism Problem* for **directed multigraphs**. The objective is to determine whether a given multigraph  $G$  can be mapped at least  $k$  distinct times to another multigraph  $H$ . If this condition is not satisfied, the algorithm should compute the *minimal extension* of  $H$  such that the extended graph contains at least  $k$  distinct subgraphs that are isomorphic to  $G$ .

The report introduces formal definitions of the **size of a multigraph** and a suitable **metric** in the space of multigraphs, which are necessary for analyzing the problem. It then presents an algorithm that constructs the minimal extension of  $H$ . Finally, the computational properties of the proposed algorithm are examined, and a polynomial approximation is discussed in cases where the exact approach exhibits exponential complexity.

## 2 Definition of Graph Size

Throughout this work, all graphs are represented using an **adjacency matrix** representation. For a directed multigraph with vertex set  $V = \{v_1, \dots, v_n\}$ , the adjacency matrix  $A$  has dimensions  $n \times n$ , and each entry  $A[i][j] = w$  denotes the number  $w \geq 0$  of directed edges from vertex  $v_i$  to vertex  $v_j$ . This representation naturally captures edge multiplicity while maintaining a fixed matrix size determined solely by the vertex count.

The size of a graph is not formally defined in a universal way, but for the purposes of this work we **define** it as:

$$|G| = |V(G)|$$

that is, the total number of vertices in the graph. This choice is motivated by the fact that both the memory requirements and the computational complexity of the algorithms depend primarily on the number of vertices, as the adjacency matrix grows quadratically with this parameter ( $n \times n$ ). Thus, the vertex count provides a consistent and practical measure of graph size for all analyses and computations presented in this project.

## 3 Metric in the Space of Graphs

To reason about the *minimality* of an extension of a graph  $H$ , a suitable distance measure between graphs is required. In this work, we employ the **Graph Edit Distance (GED)** as the metric for directed multigraphs represented by adjacency matrices.

The GED quantifies the minimum total cost of a sequence of edit operations that transform one graph into another. The allowed edit operations are:

- **Vertex insertion or deletion**, which adds or removes a vertex along with the corresponding row and column in the adjacency matrix.
- **Edge multiplicity increment or decrement**, which increases or decreases the value of  $A[i][j]$  by one.

Each edit operation has an associated strictly positive cost  $\mathcal{C}(e) > 0$ . An *edit path*  $\pi$  is a finite sequence of such edits, with total cost  $\text{cost}(\pi) = \sum_{e \in \pi} \mathcal{C}(e)$ . The Graph Edit Distance between two graphs  $G$  and  $H$  is defined as:

$$d_{\text{GED}}(G, H) = \min_{\pi: G \rightarrow H} \text{cost}(\pi).$$

**GED is a Metric.** We verify that  $d_{\text{GED}}$  satisfies the four metric axioms for all directed multigraphs under the above edit model:

1. **Non-negativity:** Each edit has a nonnegative cost, so  $d_{\text{GED}}(G, H) \geq 0$ .
2. **Identity of indiscernibles:**  $d_{\text{GED}}(G, H) = 0$  if and only if  $G = H$ , since only the empty edit sequence can have zero cost.

3. **Symmetry:** Each edit has an inverse operation with equal cost, therefore  $d_{\text{GED}}(G, H) = d_{\text{GED}}(H, G)$ .
4. **Triangle inequality:** Concatenating optimal edit paths from  $G$  to  $K$  and from  $K$  to  $H$  yields a valid path from  $G$  to  $H$ , giving

$$d_{\text{GED}}(G, H) \leq d_{\text{GED}}(G, K) + d_{\text{GED}}(K, H).$$

**Relevance to Minimal Extensions.** In this framework, extending  $H$  minimally to satisfy the  $k$ -subgraph isomorphism condition corresponds to finding a graph  $H'$  that admits at least  $k$  distinct embeddings of  $G$  while minimizing  $d_{\text{GED}}(H, H')$ . Because the GED is defined directly in terms of adjacency matrix edits, it provides a natural, quantitative, and computationally compatible measure of distance in the space of directed multigraphs.

## 4 Exact Minimal Extension for $k$ -Subgraph Isomorphism

**Mathematical formulation.** Let  $G = (V_G, E_G)$  and  $H = (V_H, E_H)$  be directed multigraphs with adjacency matrices  $A_G$  and  $A_H$ , respectively. A **mapping** from  $G$  to  $H$  is an **injective function**

$$f : V_G \rightarrow V_H,$$

such that for every pair of vertices  $(u, v) \in V_G \times V_G$  with  $A_G(u, v) > 0$ , the corresponding edge multiplicity in  $H$  after extension satisfies

$$A_H(f(u), f(v)) \geq A_G(u, v).$$

If this condition holds,  $f$  defines an *embedding* of  $G$  into  $H$ . The goal of the algorithm is to find a minimal extension  $H'$  of  $H$  such that there exist at least  $k$  distinct injective mappings  $f_1, f_2, \dots, f_k : V_G \rightarrow V_{H'}$  satisfying the above inequality for  $A_{H'}$ .

**High-level idea.** Building upon the classical subgraph isomorphism search procedure [1], the proposed method must account for the specific requirements of *multigraphs*, where multiple edges can exist between the same pair of vertices.

The algorithm proceeds in three main steps:

1. **Enumeration of Injective Mappings:** We first generate all possible injective mappings from the vertices of  $G$  to the vertices of  $H$ . Unlike simple graphs, checking for valid isomorphism candidates here involves comparing edge counts (multiplicities) rather than simple existence.
2. **Subset Selection:** We iterate through all possible subsets of size  $k$  from the generated mappings. Each subset represents a potential set of  $k$  distinct embeddings.

3. **Cost Calculation and Minimization:** For each subset of  $k$  mappings, we calculate the cost of extending  $H$  to support all  $k$  embeddings simultaneously. For a specific edge  $(u, v)$  in  $G$  with multiplicity  $w_G$ , and a mapping  $f$ , the target edge in  $H$  is  $(f(u), f(v))$ . If multiple mappings in the chosen subset map different edges of  $G$  to the same edge in  $H$ , we must ensure  $H$  has enough capacity to satisfy the *maximum* demand among them, rather than the sum. The cost is the sum of added edge multiplicities required to satisfy all mappings in the subset. We select the subset that minimizes this total cost.

**Correctness.** The algorithm exhaustively explores the solution space by generating every possible combination of  $k$  vertex mappings. For each combination, it calculates the minimal necessary edge additions to make those mappings valid. By taking the global minimum over all combinations, the algorithm guarantees finding the extension with the lowest Graph Edit Distance. The correct handling of multigraphs is ensured by calculating the cost based on edge multiplicity differences:  $\max(0, A_G[u, v] - A_H[f(u), f(v)])$ .

**Complexity.** Let  $n_G = |V(G)|$ ,  $n_H = |V(H)|$ ,  $e_G$  be the number of edges in  $G$ , and  $e_H$  be the number of edges in  $H$ .

1. **Mapping Enumeration:**

The first vertex of  $G$  can be mapped to any of the  $n_H$  vertices of  $H$ , the second to one of the remaining  $n_H - 1$ , and so on, until all  $n_G$  vertices are mapped. Therefore, the total number of distinct injective mappings is the number of ordered selections of  $n_G$  vertices from  $n_H$ :

$$P(n_H, n_G) = n_H \times (n_H - 1) \times \cdots \times (n_H - n_G + 1) = \frac{n_H!}{(n_H - n_G)!}.$$

2. **Cost Calculation per Subset:** The algorithm iterates over all  $\binom{P(n_H, n_G)}{k}$  subsets of mappings. For a fixed subset of size  $k$ , we must compute the cost. Crucially, because we are dealing with multigraphs, the distinctness of an embedding depends not just on vertex mapping but on which specific edges are used. However, our problem formulation defines distinctness via the vertex mapping functions  $f$ . Calculating the cost for a subset involves iterating over all edges in  $G$  for each mapping to determine the necessary increments in  $H$ . This takes  $O(k \cdot n_G^2)$ .

The overall time complexity is dominated by the number of subsets:

$$O\left(\binom{\frac{n_H!}{(n_H - n_G)!}}{k} \cdot k \cdot n_G^2\right)$$

This is factorial in  $n_H$ , making the exact approach feasible only for very small graphs and small values of  $k$ .

---

**Algorithm 1** Exact Algorithm for Minimal  $k$ -Subgraph Extension (Multigraph)

---

**Require:** Multigraphs  $G = (V_G, E_G)$ ,  $H = (V_H, E_H)$ , integer  $k$

**Ensure:** Multiset of additional edges  $\mathcal{E}_{\text{add}}$  required to extend  $H$

```
1:  $A_G \leftarrow \text{AdjacencyMatrix}(G)$ 
2:  $A_H \leftarrow \text{AdjacencyMatrix}(H)$ 
3:  $\mathcal{M} \leftarrow \emptyset$   $\triangleright$  List of all injective vertex mappings  $V_G \rightarrow V_H$ 
4: function ENUMERATEMAPPINGS( $V_G, V_H, mapping, used\_in\_H$ )
5:   if  $|mapping| = |V_G|$  then
6:      $\mathcal{M}.\text{add}(\text{COPY}(mapping))$ 
7:   return
8:   end if
9:    $u \leftarrow$  next vertex of  $V_G$ 
10:  for all  $v \in V_H$  do
11:    if  $v \notin used\_in\_H$  then
12:       $mapping[u] \leftarrow v$ 
13:       $used\_in\_H.\text{add}(v)$ 
14:      ENUMERATEMAPPINGS( $V_G, V_H, mapping, used\_in\_H$ )
15:       $used\_in\_H.\text{remove}(v)$ 
16:       $mapping.\text{remove}(u)$   $\triangleright$  Backtrack
17:    end if
18:  end for
19: end function

20: ENUMERATEMAPPINGS( $V_G, V_H, \emptyset, \emptyset$ )
21:  $minTotalCost \leftarrow \infty$ 
22:  $bestExtension \leftarrow \emptyset$ 
23: for all  $T \in \text{COMBINATIONS}(\mathcal{M}, k)$  do
24:    $currentExtension \leftarrow$  matrix of zeros like  $A_H$ 
25:    $tempH \leftarrow \text{COPY}(A_H)$ 
26:    $subsetCost \leftarrow 0$ 
27:    $\triangleright$  Calculate required edge additions for this set of  $k$  mappings
28:   for all  $f \in T$  do
29:     for all  $(u, v) \in V_G \times V_G$  do
30:       if  $A_G[u][v] > 0$  then
31:          $x \leftarrow f(u)$ 
32:          $y \leftarrow f(v)$ 
33:          $needed \leftarrow A_G[u][v]$ 
34:          $existing \leftarrow tempH[x][y]$ 
35:         if  $existing < needed$  then
36:            $diff \leftarrow needed - existing$ 
37:            $tempH[x][y] \leftarrow tempH[x][y] + diff$ 
38:            $currentExtension[x][y] \leftarrow currentExtension[x][y] + diff$ 
39:            $subsetCost \leftarrow subsetCost + diff$ 
40:         end if
41:       end if
42:     end for
43:   end for
44:   if  $subsetCost < minTotalCost$  then
45:      $minTotalCost \leftarrow subsetCost$ 
46:      $bestExtension \leftarrow currentExtension$ 
47:   end if
48: end for
return  $bestExtension$ 
```

---

## 5 Polynomial Approximation

**High-level idea.** To overcome the factorial complexity of the exact algorithm, we propose a greedy polynomial-time heuristic that determines similarity between vertices based on their in-degrees and out-degrees. Instead of searching for the global optimum among all subsets of mappings, we iteratively find and fix  $k$  distinct embeddings one by one.

The algorithm performs the following steps  $k$  times:

1. **Cost Matrix Construction:** We build a cost matrix  $C$  where  $C[u][v]$  represents the dissimilarity between vertex  $u \in V_G$  and  $v \in V_H$ . This dissimilarity is based on the difference in degrees. Crucially, to handle the "distinct mappings" constraint, we penalize pairs that have been used frequently in previous iterations or force the algorithm to find a different permutation.
2. **Optimal Assignment:** We solve the Linear Assignment Problem (LAP) using the **Hungarian Algorithm** [2] on matrix  $C$  to find the best injective mapping  $f : V_G \rightarrow V_H$  for the current iteration.
3. **Graph Update:** We immediately apply this mapping to  $H$ . If the mapping  $f$  requires edges that do not exist in  $H$ , we add them to  $H$ . This updates the topology of  $H$  for the next iteration, effectively filling in the graph so subsequent embeddings might be cheaper.

**Correctness.** While this greedy strategy does not guarantee the global minimum GED, it ensures that we produce a valid solution where  $H'$  contains at least  $k$  embeddings of  $G$ . The distinctness of embeddings is managed by tracking used assignments or modifying the graph structure such that subsequent best-matches naturally diverge or utilize the newly added capacity. The use of edge multiplicities in the degree calculation ensures that the multigraph nature is respected during the similarity matching.

**Complexity.** Let  $n_G = |V_G|$  and  $n_H = |V_H|$ . In each of the  $k$  iterations, the algorithm first computes the in- and out-degree vectors for all vertices in both  $G$  and  $H$ , which takes  $O(n_G^2 + n_H^2)$  time. It then constructs a neighborhood similarity cost matrix of size  $n_G \times n_H$  in  $O(n_G \cdot n_H)$  time. The optimal injective mapping is computed using the Hungarian algorithm, which runs in  $O(n_H^3)$ , and finally the required edge insertions are applied to the graph in  $O(n_G^2)$  time. Therefore, the overall runtime of the algorithm is  $O(k \cdot n_H^3)$ , which is polynomial in the size of the input graphs.

---

**Algorithm 2** Greedy Approximation for  $k$ -Subgraph Embedding (Multigraph)

---

**Require:** Multigraphs  $G = (V_G, E_G)$ ,  $H = (V_H, E_H)$ , integer  $k$

**Ensure:** Modified graph  $H'$  with at least  $k$  embeddings of  $G$

```
1: function DEGREE SIGNATURE( $A, v$ )
2:   return  $(\sum_j A[v][j], \sum_i A[i][v])$   $\triangleright$  Out- and in-degree
3: end function
4:  $A\_H\_current \leftarrow \text{COPY}(A_H)$ 
5:  $FoundEmbeddings \leftarrow \emptyset$ 
6: for  $i = 1$  to  $k$  do
7:   Initialize cost matrix  $C$  of size  $|V_G| \times |V_H|$ 
8:   for all  $u \in V_G$  do
9:     for all  $v \in V_H$  do
10:       $(d_{out}^G, d_{in}^G) \leftarrow \text{DEGREE SIGNATURE}(A_G, u)$ 
11:       $(d_{out}^H, d_{in}^H) \leftarrow \text{DEGREE SIGNATURE}(A_H, v)$ 
12:       $C[u][v] \leftarrow |d_{out}^G - d_{out}^H| + |d_{in}^G - d_{in}^H|$ 
13:       $\triangleright$  Heuristic to encourage distinct mappings if necessary
14:      if Strategy to avoid exact previous mapping then
15:         $C[u][v] \leftarrow C[u][v] + \text{Penalty}$ 
16:      end if
17:    end for
18:   $f \leftarrow \text{HUNGARIAN ALGORITHM}(C)$   $\triangleright$  Returns map  $V_G \rightarrow V_H$ 
19:   $\triangleright$  Extend H to support this specific mapping
20:  for all  $u \in V_G$  do
21:    for all  $w \in V_G$  do
22:      if  $A_G[u][w] > 0$  then
23:         $x \leftarrow f(u)$ 
24:         $y \leftarrow f(w)$ 
25:         $needed \leftarrow A_G[u][w]$ 
26:         $have \leftarrow A\_H\_current[x][y]$ 
27:        if  $have < needed$  then
28:           $diff \leftarrow needed - have$ 
29:           $A\_H\_current[x][y] \leftarrow A\_H\_current[x][y] + diff$ 
30:        end if
31:      end if
32:    end for
33:   $FoundEmbeddings.add(f)$ 
34: end for
35: return  $A\_H\_current$ 
```

---

---

**Algorithm 3** Hungarian Algorithm for Minimum-Cost Injective Assignment

---

**Require:** Cost matrix  $C[u][v]$  for  $u \in V_G, v \in V_H$

**Ensure:** Injective mapping  $f : V_G \rightarrow V_H$  minimizing total assignment cost

```
1:  $n \leftarrow |V_G|, m \leftarrow |V_H|$ , where  $n \leq m$ 
2: Initialize dual variables  $uLabels[u] \leftarrow 0, vLabels[v] \leftarrow \min_u C[u][v]$ 
3: Initialize matchings  $f : V_G \rightarrow \text{None}, f^{-1} : V_H \rightarrow \text{None}$ 
4: for all  $u \in V_G$  do
5:   Initialize  $minSlack[v] \leftarrow \infty, prev[v] \leftarrow \text{None}$ 
6:   Initialize visited sets  $S \leftarrow \{u\}, T \leftarrow \emptyset$ 
7:    $parent[u] \leftarrow \text{None}, queue \leftarrow [u]$ 
8:   while matching not complete do
9:     Dequeue  $u'$  from  $queue$ 
10:    for all  $v \in V_H \setminus T$  do
11:       $\delta \leftarrow C[u'][v] - uLabels[u'] - vLabels[v]$ 
12:      if  $\delta < minSlack[v]$  then
13:         $minSlack[v] \leftarrow \delta, prev[v] \leftarrow u'$ 
14:      end if
15:    end for
16:    Let  $v^* = \arg \min_v minSlack[v], \Delta = minSlack[v^*]$ 
17:    for all  $u' \in S$  do
18:       $uLabels[u'] \leftarrow uLabels[u'] + \Delta$ 
19:    end for
20:    for all  $v \in T$  do
21:       $vLabels[v] \leftarrow vLabels[v] - \Delta$ 
22:    end for
23:    for all  $v \notin T$  do
24:       $minSlack[v] \leftarrow minSlack[v] - \Delta$ 
25:    end for
26:    Add  $v^*$  to  $T$ 
27:    if  $f^{-1}[v^*] = \text{None}$  then ▷ Found augmenting path
28:      Backtrack to build path using  $prev[v^*]$ 
29:      while  $v^* \neq \text{None}$  do
30:         $u' \leftarrow prev[v^*]$ 
31:        Swap:  $f[u'] \leftarrow v^*, f^{-1}[v^*] \leftarrow u'$ 
32:         $v^* \leftarrow old\ f[u']$ 
33:      end while
34:      Break
35:    else
36:       $u'' \leftarrow f^{-1}[v^*]$ 
37:      Add  $u''$  to  $S$ , enqueue  $u''$ 
38:      Update  $parent[u''] \leftarrow v^*$ 
39:    end if
40:  end while
41: end for
42: return mapping  $f$ 
```

---

## 6 Experiments

To evaluate the practical performance of both algorithms, we conducted extensive experiments on randomly generated directed multigraphs. The experiments compare the **solution quality** (accuracy) and **execution time** of the exact and approximation algorithms.

### 6.1 Experimental Setup

**Test Instance Generation.** We generated 4,950 test cases with varying parameters:

- **Pattern graph  $G$ :** sizes  $n_G \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15\}$
- **Host graph  $H$ :** sizes determined by ratios  $n_H/n_G \in \{1.5, 2.0, 2.5, 3.0, 4.0\}$
- **Parameter  $k$ :** values from 1 to 10
- **Edge density:** sparse (0.2), medium (0.5), and dense (0.8)
- **Edge multiplicity:** random values in  $\{0, 1, 2, 3\}$

For each parameter combination, 3 random instances were generated using different seeds.

**Execution Environment.** The exact algorithm was given a timeout of 300 seconds per instance. Instances exceeding this limit, or having estimated complexity beyond the first observed timeout threshold, were skipped using an adaptive complexity-based pruning strategy.

### 6.2 Time Complexity Analysis

**Exact Algorithm.** The execution time of the exact algorithm grows rapidly with both  $n_G$  and  $n_H$ , confirming the factorial complexity derived in Section 4. Table 1 summarizes the average execution times.

| $n_G$ | Mean Time (s) | Instances |
|-------|---------------|-----------|
| 2     | 0.37          | 306       |
| 3     | 0.50          | 117       |
| 4     | 3.41          | 54        |
| 5     | 31.09         | 27        |
| 6     | 33.94         | 9         |

Table 1: Average execution time of the exact algorithm by pattern graph size.

The exponential growth is clearly visible: increasing  $n_G$  from 4 to 5 results in approximately a 10-fold increase in execution time. For  $n_G \geq 6$ , most instances either timed out or were skipped due to prohibitive complexity.

**Approximation Algorithm.** In contrast, the approximation algorithm exhibits stable polynomial-time behavior across all tested graph sizes, as shown in Table 2.

| $n_G$ | Mean Time (ms) | Instances |
|-------|----------------|-----------|
| 2–6   | 2.8–2.9        | 450 each  |
| 7–10  | 2.9–3.4        | 450 each  |
| 12    | 3.8            | 450       |
| 15    | 4.6            | 450       |

Table 2: Average execution time of the approximation algorithm.

The approximation algorithm maintains sub-5ms execution times even for  $n_G = 15$ , representing a **speedup of over 10,000×** compared to the exact algorithm on instances where both complete.

### 6.3 Solution Quality Analysis

**Approximation Ratio.** We define the approximation ratio as  $\rho = \text{cost}_{\text{approx}} / \text{cost}_{\text{exact}}$ , where cost denotes the Graph Edit Distance (number of edges added). A ratio of 1.0 indicates an optimal solution. Table 3 summarizes the results across 513 paired comparisons.

| $n_G$          | Mean $\rho$ | Median $\rho$ | Optimal (%)  |
|----------------|-------------|---------------|--------------|
| 2              | 1.83        | 1.00          | 53.7%        |
| 3              | 2.45        | 2.00          | 24.1%        |
| 4              | 3.87        | 3.40          | 19.0%        |
| 5              | 6.33        | 4.33          | 5.6%         |
| 6              | 2.65        | 2.56          | 0.0%         |
| <b>Overall</b> | <b>2.35</b> | <b>1.36</b>   | <b>26.1%</b> |

Table 3: Approximation quality by pattern graph size.

#### Key Observations.

1. The approximation algorithm finds the **optimal solution in 26.1%** of all cases.
2. For small graphs ( $n_G = 2$ ), the algorithm achieves optimal solutions in over half the instances.
3. The median approximation ratio of 1.36 indicates that in typical cases, the approximation adds only about 36% more edges than necessary.
4. The mean ratio of 2.35 is influenced by outliers where the greedy heuristic makes suboptimal initial choices.

## 6.4 Trade-off Summary

Figure 1 and Figure 2 illustrate the fundamental trade-off between the two algorithms.

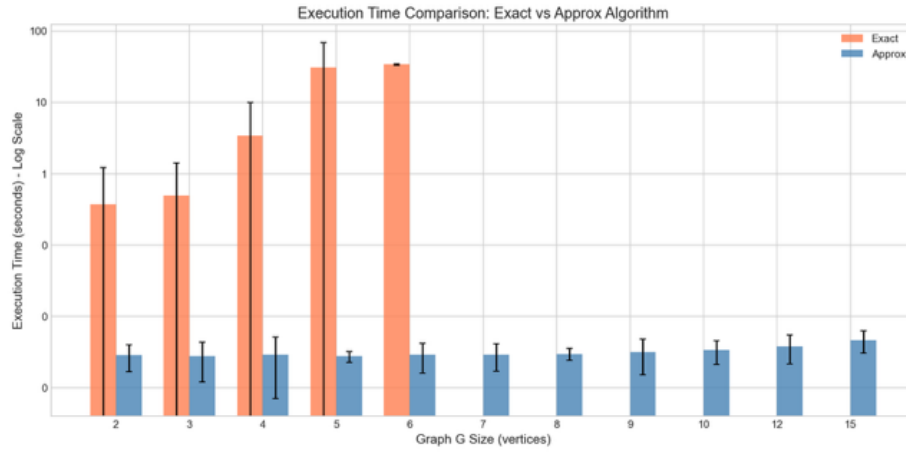


Figure 1: Execution time comparison (log scale) showing exact vs. approximation solutions.

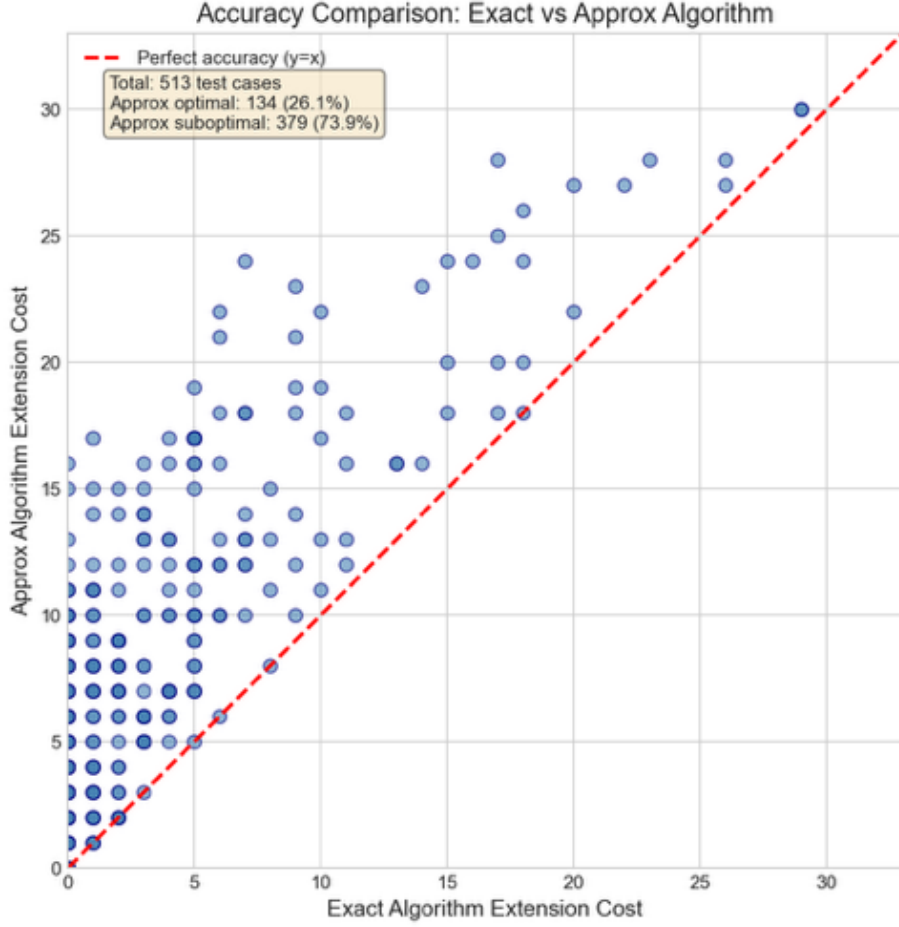


Figure 2: Extension cost comparison showing exact vs. approximation solutions.

The experimental results confirm the theoretical analysis:

- The **exact algorithm** guarantees optimal solutions but becomes impractical for  $n_G > 5$  due to factorial complexity.
- The **approximation algorithm** provides solutions within a factor of 2–3 of optimal in polynomial time, making it suitable for larger instances where the exact approach is infeasible.

## References

- [1] J. R. Ullmann, “An algorithm for subgraph isomorphism,” *Journal of the ACM*, vol. 23, no. 1, pp. 31–42, 1976.

- [2] H. W. Kuhn, “The hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.